Cyber Mind Space
Module 05 - Vulnerability Analysis

Overview

Vulnerability Analysis is a systematic process of identifying, quantifying, and prioritizing security vulnerabilities in computer systems, networks, and applications. It involves comprehensive examination, discovery, and assessment of security weaknesses that could be exploited by attackers. This module covers vulnerability assessment methodologies, scoring systems, common vulnerabilities databases, and automated scanning tools used to detect and assess security flaws.

Learning Objectives

- Understand vulnerability assessment lifecycle and methodologies
- Master vulnerability scoring systems (CVSS) and classification
- Learn to use various vulnerability scanning tools effectively
- Develop skills in manual vulnerability testing techniques
- Create comprehensive vulnerability assessment reports

Fundamentals of Vulnerability Assessment

What is Vulnerability Assessment?

Vulnerability Assessment is a process of examination, discovery, and identification of a system and applications security measures and weakness. It helps to recognize the vulnerabilities that could be exploited, need of additional security layers, and information that can be revealed using scanners.

Vulnerability Assessment Life-Cycle

- Discover - Discover vulnerabilities in systems and applications
- Prioritize - Prioritize vulnerabilities based on target, impact, loss value, business impact, etc.
- Report - Prepare comprehensive reports on vulnerabilities
- Remediate - Take appropriate preventive actions
- Verify - Verify that vulnerabilities have been resolved
- Monitor - Monitor network traffic and system behaviors for further intrusions

Vulnerability Scoring Systems

CVSS (Common Vulnerability Scoring System)

CVSS v2.0 Scoring

- Low: 0.0 - 3.9
- Medium: 4.0 - 6.9
- High: 7.0 - 10.0

CVSS v3.0/3.1 Scoring

- None: 0.0 (INFO)
- Low: 0.1 - 3.9
- Medium: 4.0 - 6.9
- High: 7.0 - 8.9
- Critical: 9.0 - 10.0

CVSS Metrics

Base Metrics

- Attack Vector (AV): Network, Adjacent Network, Local, Physical
- Attack Complexity (AC): Low, High
- Privileges Required (PR): None, Low, High
- User Interaction (UI): None, Required
- Scope (S): Unchanged, Changed
- Confidentiality Impact (C): None, Low, High
- Integrity Impact (I): None, Low, High
- Availability Impact (A): None, Low, High

Temporal Metrics

- Exploit Code Maturity: Not Defined, Unproven, Proof-of-Concept, Functional, High
- Remediation Level: Not Defined, Official Fix, Temporary Fix, Workaround, Unavailable
- Report Confidence: Not Defined, Unknown, Reasonable, Confirmed

Environmental Metrics

- Confidentiality Requirement: Not Defined, Low, Medium, High
- Integrity Requirement: Not Defined, Low, Medium, High
- Availability Requirement: Not Defined, Low, Medium, High

Cyber Mind Space

Common Vulnerabilities and Exposures (CVE)

CVE Database Resources

Whenever a vulnerability is exposed globally, it is distinguished by its CVE number.

Primary CVE Platforms:

- **National Vulnerability Database (NVD)**: *https://nvd.nist.gov/*
- **MITRE CVE**: *https://cve.mitre.org/*
- **Exploit Database**: *https://exploit-db.com/*
- **CVE Details**: *https://www.cvedetails.com/*
- **Vulnerability Lab**: *https://vuldb.com/*

CWE (Common Weakness Enumeration)

CWE-79: Cross-site Scripting (XSS)
CWE-89: SQL Injection
CWE-119: Buffer Overflow
CWE-200: Information Exposure
CWE-264: Permissions, Privileges, and Access Controls
CWE-310: Cryptographic Issues
CWE-352: Cross-Site Request Forgery (CSRF)
CWE-434: Unrestricted Upload of File with Dangerous Type

Vulnerability Scanners and Tools

Network Vulnerability Scanners

Nmap (Network Mapper) with NSE Scripts

Nmap's Scripting Engine provides powerful vulnerability detection capabilities.

Basic Vulnerability Scanning:

```
# General vulnerability scan
nmap --script vuln target-ip

# Safe vulnerability scripts only
nmap --script safe target-ip

# Specific vulnerability categories
nmap --script auth target-ip nmap -                #Authentication bypass
-script brute target-ip nmap --                    #Brute force attacks
script discovery target-ip nmap --                 #Discovery scripts
script dos target-ip nmap --script                 #Denial of service
exploit target-ip nmap --script                    #Exploitation scripts
external target-ip nmap --script                   #External scripts
fuzzer target-ip nmap --script                     #Fuzzing scripts
intrusive target-ip nmap --script                  #Intrusive tests
malware target-ip nmap --script                    #Malware detection
version target-ip                                  #Version detection
```

Protocol-Specific Vulnerability Testing:

```
# SMB vulnerabilities nmap --script smb-
vuln-* target-ip nmap --script smb-vuln-
ms17-010 target-ip nmap --script smb-vuln-         # EternalBlue
ms08-067 target-ip nmap --script smb-vuln-         # MS08-067
conficker target-ip                                # Conficker

# SSL/TLS vulnerabilities
nmap --script ssl-* target-ip
nmap --script ssl-heartbleed target-ip             # Heartbleed
nmap --script ssl-poodle target-ip                 # POODLE
nmap --script ssl-ccs-injection target-ip          #CCS Injection

# Web application vulnerabilities
nmap --script http-vuln-* target-ip
nmap --script http-shellshock target-ip            # Shellshock
nmap --script http-slowloris-check target-ip       #Slowloris DoS

# Database vulnerabilities
nmap --script mysql-vuln-cve2012-2122target-ip    #MySQL auth bypass
nmap --script ms-sql-info target-ip                #MS SQL info
```

Nessus Professional

Cyber Mind Space

```
# Nessus command line interface (if available)
nessuscli scan new --name "Vulnerability Scan" --targets "192.168.1.0/24" --template "basic"
nessuscli scan launch "scan-id"
nessuscli scan export "scan-id" --format nessus --output report.nessus
```

Nessus Policy Templates:

- **Basic Network Scan**: General vulnerability assessment
- **Advanced Scan**: Comprehensive vulnerability testing
- **Web Application Tests**: Focused web application scanning
- **Malware Scan**: Malware and backdoor detection
- **Mobile Device Scan**: Mobile device vulnerability assessment
- **SCADA Audit**: Industrial control system assessment

OpenVAS

```
#  OpenVAS  setupandscanning
sudo  gvm-setup  sudo  gvm-                          #Initial setup
start    sudo    gvm-create-                         #Start services
useradmin                                            #Create admin user

# Command line scanning with gvm-cli
gvm-cli socket --xml "<get_configs/>"                # List scan configs
gvm-cli socket --xml "<create_target><name>Target</name><hosts>192.168.1.100</hosts></create_target>"
gvm-cli socket --xml "<create_task><name>Scan</name><config id='config-id'><target id='target-id'/></create_task>"
gvm-cli socket --xml "<start_task task_id='task-id'/>"
```

Web Application Vulnerability Scanners

Nikto - Web Server Scanner

```
# Basic web server scanning
nikto -h http://target-ip                            #Basic scan
nikto -h https://target-ip-ssl                       #SSL scan
nikto -h http://target-ip-port80,443,8080,8443      #Multiple ports

# Advanced Nikto options
nikto -h http://target-ip -Tuning 123456789a         # All test types
nikto -h http://target-ip -evasion 12345678          # Evasion techniques
nikto -h http://target-ip -Format htm -output report.html # HTML output

# Specific testing categories nikto
-h  http://target-ip  -Tuning1  nikto                #Interesting files
-h  http://target-ip  -Tuning2  nikto                # Misconfiguration
-h  http://target-ip  -Tuning3  nikto                #Information disclosure
-h  http://target-ip  -Tuning4  nikto                # Injection
-h  http://target-ip  -Tuning5  nikto                #Remote file retrieval
-h  http://target-ip  -Tuning6  nikto                #Denial of service
-h  http://target-ip  -Tuning7  nikto                #Remote file retrieval
-h  http://target-ip  -Tuning8  nikto                #Command execution
-h  http://target-ip  -Tuning9  nikto                #SQL injection
-h http://target-ip -Tuninga                         #Authentication bypass
```

WPScan - WordPress Security Scanner

```
#  WordPress  vulnerability  assessment  wpscan  --url
http://target-wordpress/ wpscan --url http://target-   # Basic scan
wordpress/ --enumerate u wpscan --url http://target-   # Enumerate users
wordpress/ --enumerate p wpscan --url http://target-   # Enumerate plugins
wordpress/ --enumerate t                                # Enumerate themes
wpscan --url http://target-wordpress/ --enumerate vp,vt,u # Vulnerable plugins/themes/users

# WordPress brute force attacks
wpscan --url http://target-wordpress/ --usernames admin,administrator --passwords /usr/share/wordlists/rockyou.txt
wpscan --url http://target-wordpress/ --users-list users.txt --passwords passwords.txt

# Enhanced scanning with API token
wpscan --url http://target-wordpress/ --api-token YOUR_API_TOKEN
```

OWASP ZAP (Zed Attack Proxy)

```
# ZAP command line interface
zap.sh -cmd -quickurl http://target-ip               #Quick scan
zap.sh -cmd -quickprogress http://target-ip         #Quick scan with progress

# ZAP baseline scan
zap-baseline.py -t http://target-ip                  #Baseline scan
zap-baseline.py -t http://target-ip-rreport.html    #HTML report
```

Cyber Mind Space

```
# ZAP full scan
zap-full-scan.py -t http://target-ip                    #Full active scan
```

Database Vulnerability Scanners

SQLMap - SQL Injection Scanner

```
# Basic SQL injection testing
sqlmap -u "http://target/page.php?id=1"                # Basic test
sqlmap -u "http://target/page.php?id=1" --dbs          # List databases
sqlmap -u "http://target/page.php?id=1" -D database --tables # List tables
sqlmap -u "http://target/page.php?id=1" -D database -T table --columns # List columns

# Advanced SQLMap techniques
sqlmap -u "http://target/page.php?id=1" --batch        # Non-interactive mode
sqlmap -u "http://target/page.php?id=1" --level=5 --risk=3 # Aggressive testing
sqlmap -r request.txt                                   # Test from saved request
sqlmap -u "http://target/page.php?id=1" --os-shell     # Get OS shell
```

NoSQLMap - NoSQL Injection Scanner

```
# NoSQL injection testing
python nosqlmap.py -t http://target/login -p username,password
python nosqlmap.py -t http://target/api -p json --json
```

Manual Vulnerability Testing Techniques

Protocol-Specific Testing

SMB/NetBIOS Testing

```
# SMB enumeration andvulnerabilitytesting
enum4linux target-ip                                    #Comprehensive SMB enumeration
smbclient -L //target-ip/-N                            #List shares anonymously
smbclient //target-ip/share-N                          #Access share anonymously

# SMB version detection
smbclient -L //target-ip/ -N|grep"Domain="             #Get domain info
rpcclient -U "" -N target-ip                           #RPC null session

# Test for SMB vulnerabilities nmap --
script smb-vuln-ms17-010 target-ip nmap -              #EternalBlue test
-script smb-security-mode target-ip                    #Security mode check
```

FTP Testing

```
# FTP vulnerabilitytesting
ftp target-ip                                          #Manual FTP connection
nmap --scriptftp-anontarget-ip                         #Anonymous FTP check
nmap --scriptftp-bouncetarget-ip                       #FTP bounce check
nmap --scriptftp-vsftpd-backdoortarget-ip             #VSFTPD backdoor check

# FTP brute force
hydra -l admin -P passwords.txt target-ip ftp
medusa -u admin -P passwords.txt -h target-ip -M ftp
```

SSH Testing

```
# SSH vulnerabilityassessment
ssh-audit target-ip                                    #SSH configuration audit
nmap --script ssh-hostkeytarget-ip                     #SSH host key info
nmap --script ssh-auth-methodstarget-ip                #Supported auth methods

# SSH brute force
hydra -l root -P passwords.txt target-ip ssh
medusa -u root -P passwords.txt -h target-ip -M ssh
```

Web Application Manual Testing

```
# Directory enumeration
gobuster dir -u http://target-ip -w /usr/share/wordlists/dirb/common.txt
dirbuster -u http://target-ip -l /usr/share/wordlists/dirb/big.txt

# Technology detection
whatweb http://target-ip                                #Web technology detection
wafw00f http://target-ip                               #WAF detection
```

Cyber Mind Space

```
#   SSL/TLS   testing
sslscan     target-ip                                   #SSL  configuration  scan
testssl.sh target-ip                                    #Comprehensive SSL test
```

Advanced Vulnerability Assessment Techniques

Custom Vulnerability Scripts

Nmap NSE Custom Scripts

```
-- Example custom Nmap script for vulnerability detection
local shortport = require "shortport"
local nmap = require "nmap"
local stdnse = require "stdnse"

description = [[
Custom vulnerability detection script
]]

author = "CEH Student"
license = "Same as Nmap--See https://nmap.org/book/man-legal.html"
categories = {"vuln", "safe"}

portrule = shortport.http

action=function(host, port)
    localoutput = {}

    --Custom vulnerability check logic here
    localresponse = http.get(host, port, "/admin")

    ifresponse.status == 200 then
        table.insert(output, "Admin panel accessible")
    end

    returnstdnse.format_output(true, output)
end
```

Python Vulnerability Scanner

```
#!/usr/bin/env python3
import socket
import requests
import ssl
import subprocess
from urllib.parse import urljoin

class VulnerabilityScanner:
    def __init__(self, target):
        self.target = target
        self.vulnerabilities = []

    def check_ssl_vulnerabilities(self):
        """Check for SSL/TLS vulnerabilities"""
        try:
            context = ssl.create_default_context()
            withsocket.create_connection((self.target, 443), timeout=10) as sock:
                withcontext.wrap_socket(sock, server_hostname=self.target) as ssock:
                    cert = ssock.getpeercert()
                    cipher = ssock.cipher()

                    # Check for weak ciphers
                    if cipher[1] < 128:
                        self.vulnerabilities.append(f"Weak cipher: {cipher}")

                    # Check certificate validity
                    if 'notAfter' in cert:
                        #Addcertificateexpiry check logic
                        pass

        except Exception as e:
            print(f"SSL check failed: {e}")

    def check_web_vulnerabilities(self):
        """Check for common web vulnerabilities"""
        common_paths = [
            '/admin','/administrator','/wp-admin', '/phpmyadmin',
            '/config.php','/backup','/.git', '/.env'
        ]

        for path in common_paths:
            try:
```

Cyber Mind Space

```python
                url = f"http://{self.target}{path}"
                response = requests.get(url, timeout=5)

                if response.status_code == 200:
                    self.vulnerabilities.append(f"Accessible path: {path}")

            except requests.RequestException:
                pass

    def check_service_vulnerabilities(self):
        """Check for service-specific vulnerabilities"""
        common_ports=[21,22,23,25,53,80,110,135, 139, 143, 443, 993, 995]

        for port in common_ports:
            try:
                sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
                sock.settimeout(3)
                result = sock.connect_ex((self.target, port))

                if result == 0:
                    banner=self.get_banner(self.target, port)
                    if banner:
                        # Check for known vulnerable versions
                        if "vsftpd 2.3.4" in banner.lower():
                            self.vulnerabilities.append("Vulnerable FTP version detected")

                sock.close()

            except Exception:
                pass

    def get_banner(self, host, port):
        """Get service banner"""
        try:
            sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            sock.settimeout(3)
            sock.connect((host, port))
            banner = sock.recv(1024).decode().strip()
            sock.close()
            return banner
        except:
            return None

    def generate_report(self):
        """Generate vulnerability report"""
        print(f"Vulnerability Assessment Report for {self.target}")
        print("=" * 50)

        if self.vulnerabilities:
            for vuln in self.vulnerabilities:
                print(f"[!] {vuln}")
        else:
            print("[+] No obvious vulnerabilities detected")

    def run_scan(self):
        """Run complete vulnerability scan"""
        print(f"Starting vulnerability scan of {self.target}")

        self.check_ssl_vulnerabilities()
        self.check_web_vulnerabilities()
        self.check_service_vulnerabilities()

        self.generate_report()

# Usage
if __name__ == "__main__":
    scanner = VulnerabilityScanner("example.com")
    scanner.run_scan()
```

Cyber Mind Space

Vulnerability Assessment Methodologies

OWASP Testing Guide Methodology

Information Gathering

- Conduct search engine discovery
- Fingerprint web server
- Review webserver metafiles
- Enumerate applications on webserver

Configuration and Deployment Management Testing

- Test network/infrastructure configuration
- Test application platform configuration
- Test file extensions handling

- Review old backup and unreferenced files

Identity Management Testing

- Test role definitions
- Test user registration process
- Test account provisioning process
- Test account suspension/resumption process

Authentication Testing

- Test credentials transported over encrypted channel
- Test for default credentials
- Test for weak lock out mechanism
- Test for bypassing authentication schema

Authorization Testing

- Test directory traversal/file include
- Test for bypassing authorization schema
- Test for privilege escalation
- Test for insecure direct object references

NIST SP 800-115 Testing Methodology

Planning Phase

- Define scope and objectives
- Identify testing approach
- Develop testing schedule
- Obtain proper authorization

Discovery Phase

- Network discovery
- Service discovery
- Operating system identification
- Application discovery

Attack Phase

- Vulnerability analysis
- Initial exploitation
- Expansion of influence
- Cleanup activities

Reporting Phase

- Document findings
- Risk assessment
- Remediation recommendations
- Executive summary

Cyber Mind Space

Vulnerability Assessment Tools Comparison

Tool	Type	Platform	Cost	Best For
Nessus	Network/Web	Cross-platform	Commercial	Enterprise networks
OpenVAS	Network/Web	Free	Commercial	Open source alternative
Qualys VMDR	Cloud-based	Cross-platform	Commercial	Large enterprises
Rapid7 Nexpose	Network	Cross-platform	Free	Risk management
Nikto	Web Web	Cross-platform	Free	Web servers
OWASP ZAP	Web applications	Cross-platform	Free/Commercial	Web applications
WPScan	WordPress sites	Cross-platform	Free	WordPress sites
SQLMap	Database	Cross-platform	Free	SQL injection
Burp Suite	Web Web	Cross-platform	Free/Commercial	Manual testing
Acunetix	Web applications	Windows	Commercial	Web applications

Latest Vulnerability Trends (2024)

Emerging Vulnerabilities

- **Supply Chain Attacks**: Dependencies and third-party components
- **Container Security**: Docker and Kubernetes vulnerabilities
- **Cloud Misconfigurations**: AWS, Azure, GCP security issues

- AI/ML Security: Machine learning model vulnerabilities
- IoT Device Security: Internet of Things device vulnerabilities

Recent Critical CVEs

- CVE-2023-44487: HTTP/2 Rapid Reset Attack
- CVE-2023-38831: WinRAR Code Execution Vulnerability
- CVE-2023-34362: MOVEit Transfer SQL Injection
- CVE-2023-28252: Windows Common Log File System Driver Elevation of Privilege

Zero-Day Exploitation Trends

- Browser Vulnerabilities: Chrome, Firefox, Safari zero-days
- Operating System Exploits: Windows, Linux privilege escalation
- Network Infrastructure: Router and firewall vulnerabilities
- Mobile Platforms: iOS and Android security flaws

Compliance and Standards

Industry Standards

- ISO 27001: Information Security Management Systems
- NIST Cybersecurity Framework: Identify, Protect, Detect, Respond, Recover
- PCI DSS: Payment Card Industry Data Security Standard
- HIPAA: Health Insurance Portability and Accountability Act
- SOX: Sarbanes-Oxley Act compliance requirements

Vulnerability Management Standards

- NIST SP 800-40: Creating a Patch and Vulnerability Management Program
- ISO 27005: Information Security Risk Management
- COBIT: Control Objectives for Information and Related Technologies
- FAIR: Factor Analysis of Information Risk

Vulnerability Assessment Best Practices

Pre-Assessment Preparation

- Obtain Written Authorization: Ensure proper legal documentation
- Define Scope: Clearly identify systems and networks to test
- Schedule Testing: Coordinate with system administrators
- Prepare Environment: Set up testing tools and infrastructure

During Assessment

- Start with Passive Reconnaissance: Minimize system impact
- Use Authenticated Scans: When possible for better coverage
- Monitor System Performance: Watch for service disruption
- Document Everything: Maintain detailed logs and screenshots

Post-Assessment Activities

- Verify Findings: Confirm vulnerabilities manually
- Prioritize Risks: Use CVSS scores and business impact
- Develop Remediation Plan: Provide actionable recommendations
- Create Executive Summary: High-level overview for management

Remediation Tracking

- Assign Responsibility: Clearly define who fixes what
- Set Deadlines: Establish realistic timelines
- Verify Fixes: Re-test to confirm resolution
- Update Documentation: Maintain current security posture

References and Further Reading

Official Resources

- *NIST Special Publication 800-115: Technical Guide to Information Security Testing*
- *OWASP Vulnerability Management Guide*
- *CVSS v3.1 Specification Document*
- *SANS Institute: Vulnerability Assessment*

Vulnerability Databases

Cyber Mind Space

- *National Vulnerability Database (NVD)*
- *CVE Details*
- *Exploit Database*
- *Common Weakness Enumeration (CWE)*

Training and Certification

- *SANS SEC542: Web App Penetration Testing and Ethical Hacking*
- *EC-Council Computer Hacking Forensic Investigator (CHFI)*
- *Offensive Security Certified Professional (OSCP)*

🔗 STAY CONNECTED WITH US 🔗
🔥 Stay Connected with Cyber Mind Space — Learn, Discuss & Dominate Cybersecurity! 🔐💡
📢 Telegram Channel (Updates & Resources):
https://t.me/cybermindspace
💬 Telegram Group (Ask & Network):
https://t.me/+LJvMwjAE6yA5YWQ1
📸 Instagram (Reels & Daily Tips):
https://instagram.com/cyber_ mind _space
🎥 YouTube (Podcasts, Lives & Tutorials):
https://www.youtube.com/@CyberMindSpace
🔗 LinkedIn (Professional Profile):
https://linkedin.com/in/almadadali
💻 GitHub (Tools & Scripts):
https://github.com/ALMADADALI
📲 WhatsApp Channel (Cyber Alerts):
https://whatsapp.com/channel/0029VbAzQMU9hXF70zjpCu1Y
🗣️ Discord Server (Voice & Community Chat):
https://discord.gg/qA3CqZ86hY
🎵 Tiktok:
https://www.tiktok.com/@cybermindspace
🐦 Twitter (X) – Cybersecurity Thoughts:
https://x.com/cybermindspace?s=21
⚡Subscribe, Follow & Stay Updated — Cybersecurity Never Sleeps! 🌐

Cyber Mind Space